

Deliverable 2: Detailed Project Design Document

By Ryan Beikrasouli

1. Review Your Initial Proposal:

a. Questions:

i. *Have you clearly stated your game option and its justification?*

Yes, I believe that I that in my initial deliverable I effectively and clearly stated my selected game option and the justification behind my decision.

ii. *Is your initial class diagram still valid or does it need refinement based on new ideas?*

My initial class diagram is still partly valid however it does require some changes such as adding more relationships between classes, adding the new classes and refining some of the class attributes and methods.

b. Reflect:

i. *What aspects of your initial design worked well?*

With my initial design I believe the base few classes I designed and the base functionality of the game that I was able to achieve worked extremely well and provided the base structure which I can effectively build on to expand my game in future iterations.

ii. *Which areas need more detail or clarification?*

I believe the main areas in my initial design which require more detail or clarification are better outline of the main modules I intend to use (more modularisation) and also a clearer outline as to what the purpose of each of those modules is.

2. Detailed Pseudocode:

Module buttonUtilities

```
IF __name__ == '__main__' THEN
    PRINT("Error incorrect file run please run __main__.py")
    quit()
END IF
```

```
TRY
    from world import World
    from turrets import Cannon, Machinelaser
    from constants import *
EXCEPT ModuleNotFoundError AS err
    PRINT(err)
    PRINT("Error missing module please ensure all this games modules are present
in their original directory")
    quit()
```

FUNCTION buttonSetup(doubleSpeed, levelStarted, placingTurrets, world, demoCannon, demoMachinelaser, turretType, selectedTurret)

```
IF doubleSpeed THEN
    speedButton.MouseCheck(SCREEN, newText="1x Speed")
ELSE
    speedButton.MouseCheck(SCREEN, newText="2x Speed")
END IF
```

```
IF world.money >= demoCannon.cost THEN
    buyCannonButton.MouseCheck(SCREEN)
ELSE
    buyCannonButton.MouseCheck(SCREEN, TRUE, (220, 220, 220))
```

```
    IF turretType == "cannon" THEN
        placingTurrets = FALSE
    END IF
END IF
```

```
IF world.money >= demoMachinelaser.cost THEN
    buyMachinelaserButton.MouseCheck(SCREEN)
ELSE
    buyMachinelaserButton.MouseCheck(SCREEN, TRUE, (220, 220, 220))
    IF turretType == "machinelaser" THEN
        placingTurrets = FALSE
    END IF
END IF
```

```
IF placingTurrets THEN
    cancelButton.MouseCheck(SCREEN)
ELSE
    cancelButton.active = FALSE
END IF
```

...

...

```
IF NOT levelStarted THEN
    startButton.MouseCheck(SCREEN)
ELSE
    startButton.active = False
END IF

IF selectedTurret AND selectedTurret.tier < len(TURRET_DATA) THEN
    upgradeButton.MouseCheck(SCREEN, not (world.money >=
selectedTurret.upgradeCost), (200, 200, 200), "Upgrade: $" +
str(selectedTurret.upgradeCost))
ELSE
    upgradeButton.active = False
END IF
```

```
RETURN placingTurrets
END buttonSetup
```

```
FUNCTION checkButtons(world, cursorTurret, selectedTurret, placingTurrets,
levelStarted, doubleSpeed, turretType)
```

```
IF cancelButton.MouseClick() THEN
    placingTurrets = False
END IF

IF buyCannonButton.MouseClick() THEN
    placingTurrets = TRUE
    cursorTurret = Cannon(0, 0)
    turretType = "cannon"

ELSE IF buyMachinelaserButton.MouseClick() THEN
    placingTurrets = TRUE
    cursorTurret = Machinelaser(0, 0)
    turretType = "machinelaser"
END IF

IF selectedTurret THEN
    IF upgradeButton.MouseClick() and world.money >=
selectedTurret.upgradeCost THEN
        selectedTurret.upgrade()
        world.money -= selectedTurret.upgradeCost
    END IF
END IF

IF startButton.MouseClick() THEN
    levelStarted = TRUE
END IF

IF speedButton.MouseClick() THEN
    doubleSpeed = not doubleSpeed
END IF
```

```
RETURN [placingTurrets, cursorTurret, turretType, levelStarted,
doubleSpeed]
END checkButtons
END ButtonUtilities
```

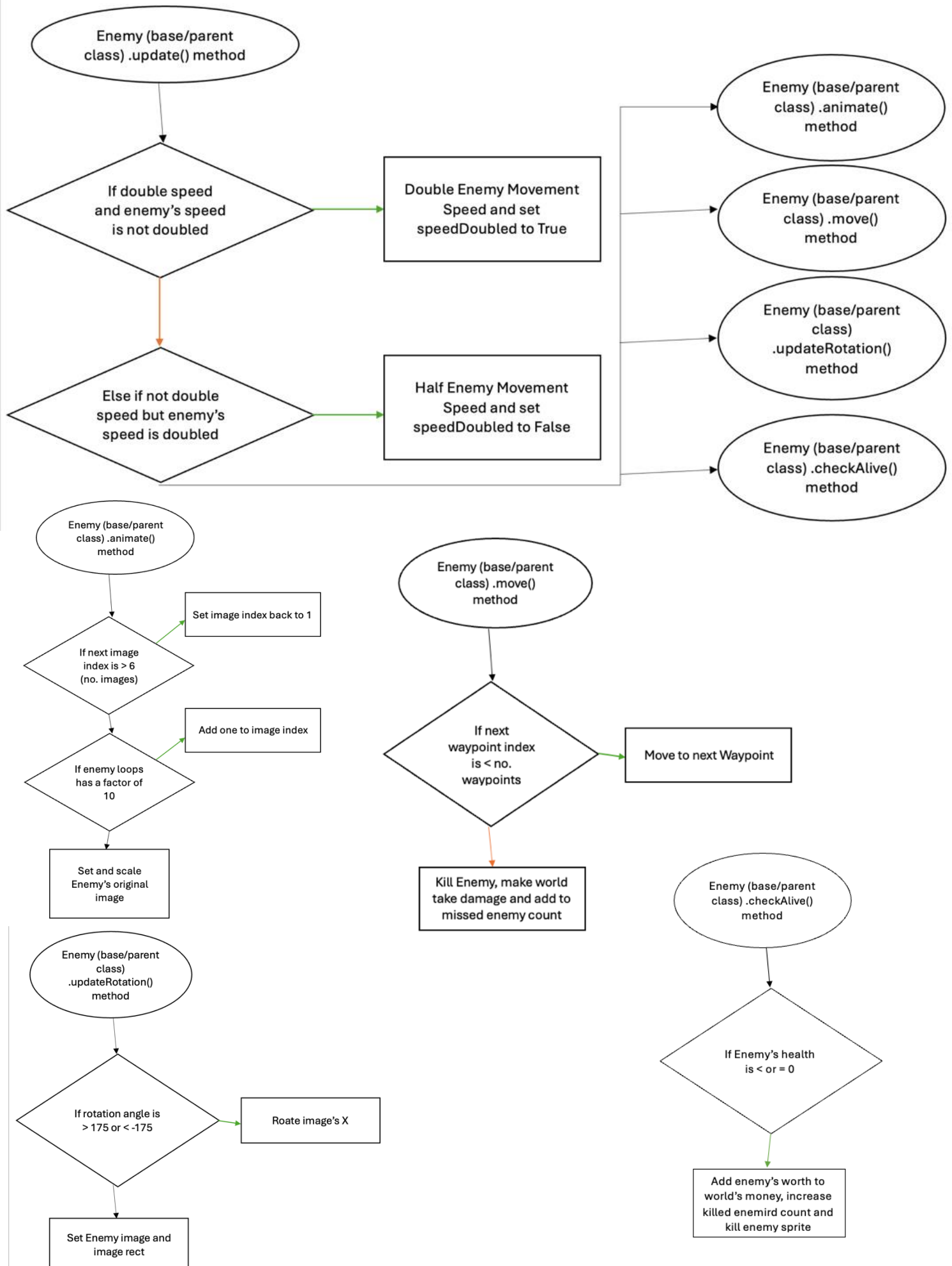
Pseudocode Analysis:

Note: variables/objects not defined in the module are defined in a separate module which is imported (mainly from the constants module)

The section highlighted in **yellow** is a section which is an example of where I have implemented error handling into my module. In that specific instance I used the error handling to handle `ModuleNotFoundError` which is raised if a module that is being imported is not found in which case the code would print the error and quit gracefully instead of crashing suddenly. My main testing method for this error exception was to randomly remove, rename or add non existent modules and observe how the errors were handled and if they were handled as expected. This final version of course was able to successfully pass my testing.

The sections highlighted in **green** show examples of OOP principles being implemented in my module. In those specific instances the `upgrade()` method is an example of inheritance as that method is inherited from the parent class `Turret` to the child classes of `Cannon` and `Machinelaser`. On the other hand `MouseCheck()` and `MouseClicked()` functions are examples of encapsulation as they allow for the use and modification of button object attributes which are all private attributes (aside from `.active`) securely and privately.

3. Flowchart:



4. Refine class diagram

```
class Button(pygame.sprite.Sprite)
- X: int
- Y: int
- Height: int
- Width: int
- squareCheck: bool
- Font: pygame.font.Font
- clicked: bool
- singleClick: bool
- textColour: Tuple[int, int, int]
+ active: bool
- Square: pygame.Rect
- SquareOnColour: Tuple[int, int, int]
- SquareOffColour: Tuple[int, int, int]
- text: str
- textRect: pygame.Rect
- text_on: pygame.surface.Surface
- text_off: pygame.surface.Surface
+ MouseCheck(SCREEN,
disabled:bool=False,
disabledColour=None, newText=None)
+ MouseClick()
```

```
class World()
+ Wave: int
+ Health: int
+ Money: int
+ enemyList: list
- worldData: dict
- image: pygame.surface.Surface
+ waypoints: list
+ tilemap: list
+ spawnedEnemies: int
+ killedEnemies: int
+ missedEnemies: int
+ processData()
- processWaypoints(data:dict)
+ processEnemies()
+ checkWaveFinished()
+ prepareNewWave()
+ draw(surface: pygame.surface.Surface)
```

```
class Cannon(Turret)
+ data: list
+ upgradeCost: int
+ cost: int
+ sfx: pygame.mixer.Sound
+ range: int
+ cooldown: int
+ damage: int
+ allTurretSheets: list
+ originalImage: pygame.surface.Surface
+ image: pygame.surface.Surface
+ rect: pygame.Rect
```

```
class MachineGuns(Turret)
+ data: list
+ upgradeCost: int
+ cost: int
+ sfx: pygame.mixer.Sound
+ range: int
+ cooldown: int
+ damage: int
+ allTurretSheets: list
+ originalImage: pygame.surface.Surface
+ image: pygame.surface.Surface
+ rect: pygame.Rect
+ allTurretSheets: list
```

```
class Turret(pygame.sprite.Sprite)
+ tileX: int
+ tileY: int
+ x: int
+ y: int
+ data: list
+ upgradeCost: int
+ cost: int
+ sfx: pygame.mixer.Sound
+ tier: int
+ range: int
+ cooldown: int
+ damage: int
+ lastShot: int
+ updateTime: int
+ selected: bool
+ target: Enemy
- doubleSpeed: bool
- upgradeSfx: pygame.mixer.Sound
+ allTurretSheets: list
+ animationImages: list
+ imageIndex: int
+ rotationAngle: float | int
+ originalImage: pygame.surface.Surface
+ image: pygame.surface.Surface
+ rect: pygame.Rect
+ rangeImage: pygame.surface.Surface
+ rangeRect: pygame.Rect
+ satUpRange()
+ update(allEnemiesGroup: pygame.sprite.Group, doubleSpeed:bool)
+ selectTarget(allEnemiesGroup: pygame.sprite.Group)
+ loadImages(spriteSheet: pygame.surface.Surface)
+ updateAnimation()
+ upgrade()
+ draw(surface: pygame.surface.Surface)
```

```
class Enemy(pygame.sprite.Sprite)
+ health: int
+ worth: int
+ damage: int
+ movementSpeed: int
+ speedDoubled: bool
+ waypoints: list
+ pos: Vector2 (similar to Tuple[int, int] | Tuple[float, float])
- nextWaypointIndex: int
+ loops: int
- rotationAngle: float
+ imageIndex: int
+ originalImage: pygame.surface.Surface
+ image: pygame.surface.Surface
+ rect: pygame.Rect
- movement: Vector2 (similar to Tuple[int, int] | Tuple[float, float])
+ update(world: World, doubleSpeed:bool)
+ move(world: World)
+ updateRotation()
+ animate()
- checkAlive(world: World)
```

```
class Skeleton(Enemy)
+ health: int
+ movementSpeed: int
+ worth: int
+ damage: int
+ originalImage: pygame.surface.Surface
+ animate()
```

```
class Zombie(Enemy)
+ health: int
+ movementSpeed: int
+ worth: int
+ damage: int
+ originalImage: pygame.surface.Surface
+ animate()
```

5. System architecture and Module Interaction:

./TowerDefence/

Deliverables	gameLoop.py	turrets.py	world.py
README.md	buttons.py	gitupdate.sh	unitTesting.py
constants.py	otherFunctions.py	__main__.py	enemies.py

./TowerDefence/Deliverables:

Deliverable 1- Project Proposal & Requirements Specification by Ryan Beikrasouli.docx

Deliverable 1- Project Proposal & Requirements Specification by Ryan Beikrasouli.pdf

Deliverable 2- Detailed Project Design Document by Ryan Beikrasouli.docx

Deliverable 2- Detailed Project Design Document by Ryan Beikrasouli.pdf

./TowerDefence/Sound Assets:

gun_shot.wav	loss_theme.mp3	upgrade_sound.wav
laser_shot.wav	main_theme.mp3	victory_theme.mp3

./TowerDefence/sprite images:

map1.json map1.png towerDefense_tilesheet.tsx

./TowerDefence/sprite images/Default Assets:

default_sprite 1.png	default_sprite 3.png	default_sprite 5.png
default_sprite 2.png	default_sprite 4.png	default_sprite 6.png

./TowerDefence/sprite images/Skeleton Assets:

skeleton 1.png skeleton 2.png skeleton 3.png skeleton 4.png

./TowerDefence/sprite images/Turret Assets:

cannon_tier_1.png	cannon_tier_4.png
machine_laser_tier_3.png	
cannon_tier_2.png	machine_laser_tier_1.png
machine_laser_tier_4.png	
cannon_tier_3.png	machine_laser_tier_2.png

./TowerDefence/sprite images/Zombie Assets:

zombie 1.png	zombie 3.png	zombie 5.png	zombie 7.png	zombie 9.png
zombie 2.png	zombie 4.png	zombie 6.png	zombie 8.png	

./TowerDefence/utilities:

__init__.py buttonUtilities.py mainUtilities.py

6. Design Decisions: Error Handling and Testing Strategies

i. Why is error handling and testing your program important?

Error handling and testing my program is important as it ensures the correct outputs and processing in my program. This is done with try catch blocks which handle errors which would cause the program to crash (such as module not found errors) and also testing which helps locate logic errors in my program.

ii. What errors is your program designed to handle and how does it handle them?

The main error's my program is designed to handle are module not found errors and file not found errors. This is done by every time I want to import a module into another module I use a try catch block to gracefully handle if the module is not found due to it being incorrectly named or moved. I also created a generic function called TryLoad() and its entire purpose is to handle the process of loading images and sounds into the program with error exceptions to gracefully handle if they are not found.

iii. How do you intend to test your program?

The main methods I intend to implement to test my program are unit tests and desk checking. For the unit testing I intend to use the pytest library which simplifies the process of unit testing my program. Then I would use it to test my program's functions and class methods to ensure they have the expected output/results. For some of my functions/methods I may also prefer to use a manual desk checking method to track specific variables on each iteration. I would then keep track of these tests through screenshots and documentation.

7. Additional Design Conciderations

i. What are some ethical/legal issues relating to your design and how do you intend to manage them?

The main ethical/legal issues relating to my design mainly involve copy right laws and the use of intellectual property. This mainly comes into play with the assets I am using in my game (such as music, sound effects and sprite images). However to manage this and abide by my ethical and legal responsibilities I have ensured that I've provided credit for the assets I got from online by crediting the original creators in my documentation (README.md file).

ii. Why did you choose your particular design over other approaches and how does your design allow for future improvements and debugging?

I decided on using a modular and OOP centered design for my game. The reason behind this is that modular and object oriented programming allows me to create clean, reusable modules/classes which simplify not only debugging by narrowing down errors and allowing for separate testing of modules. But they also allow for future scalability as their reusability allows adding new aspects to my game (for instance new enemies or turrets) much easier.